

1. Read the Maze

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

const int max_rows = 500;
const int max_columns = 500;

int main()
{
    char maze[max_rows][max_columns];
    int rows = 0;
    int columns = 0;

    ifstream fin("maze2a.txt");

    if (!fin)
    {
        cout << "Can't open file :(" << endl;
    }

    string line;

    while (getline(fin, line))
    {
        columns = line.length();

        for (int c = 0; c < columns; c++)
        {
            maze[rows][c] = line[c];
        }

        rows++;
    }

    fin.close();

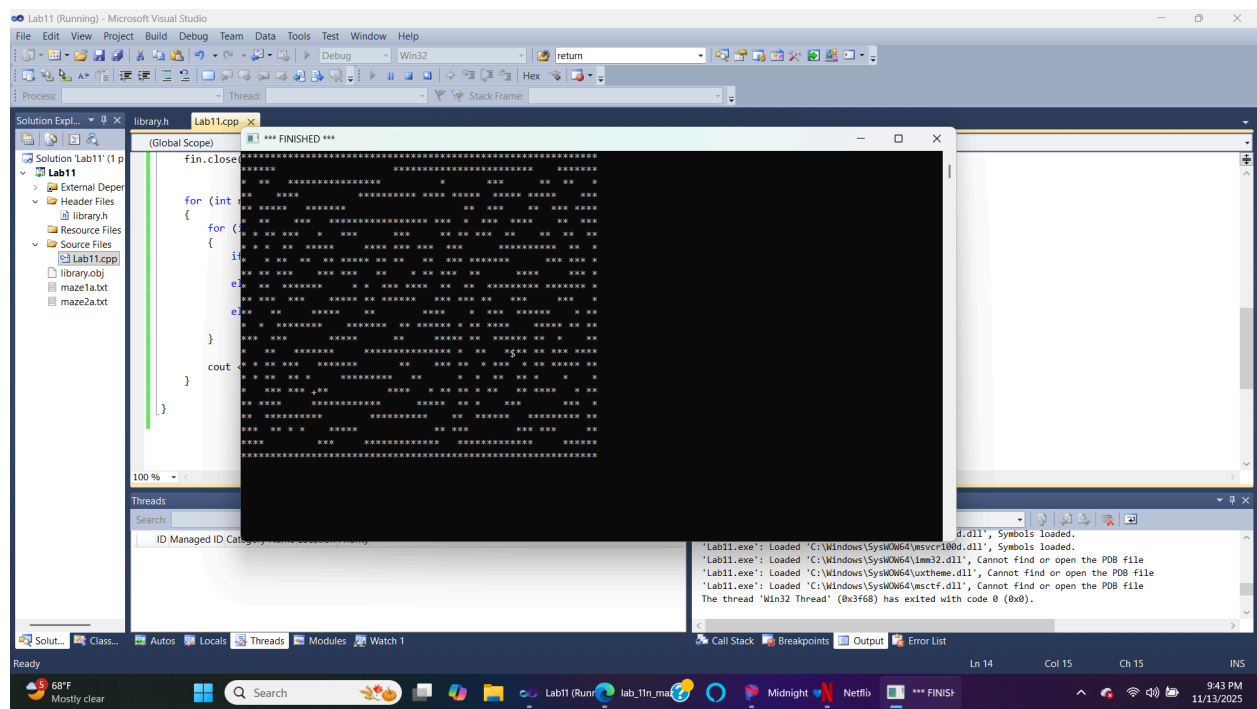
    for (int r = 0; r < rows; r++)
    {
        for (int c = 0; c < columns; c++)
```

```

    {
        if (maze[r][c] == 'X')
            cout << "*";
        else if (maze[r][c] == '-')
            cout << " ";
        else
            cout << maze[r][c];
    }

    cout << endl;
}
}

```



2. Detect + and \$

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

const int max_rows = 500;
const int max_columns = 500;

int main()
{
    char maze[max_rows][max_columns];
    int rows = 0;
    int columns = 0;
        int start_row = -1;
        int start_column = -1;
        int treasure_row = -1;
        int treasure_column = -1;

    ifstream fin("maze2a.txt");

    if (!fin)
    {
        cout << "Can't open file :(" << endl;
    }

    string line;

    while (getline(fin, line))
    {
        columns = line.length();

        for (int c = 0; c < columns; c++)
        {
            maze[rows][c] = line[c];

            if (line[c] == '+')
            {
                start_row = rows;
                start_column = c;
            }
        }
    }
}
```

```

        if (line[c] == '$')
        {
            treasure_row = rows;
            treasure_column = c;
        }
    }

    rows++;
}

fin.close();

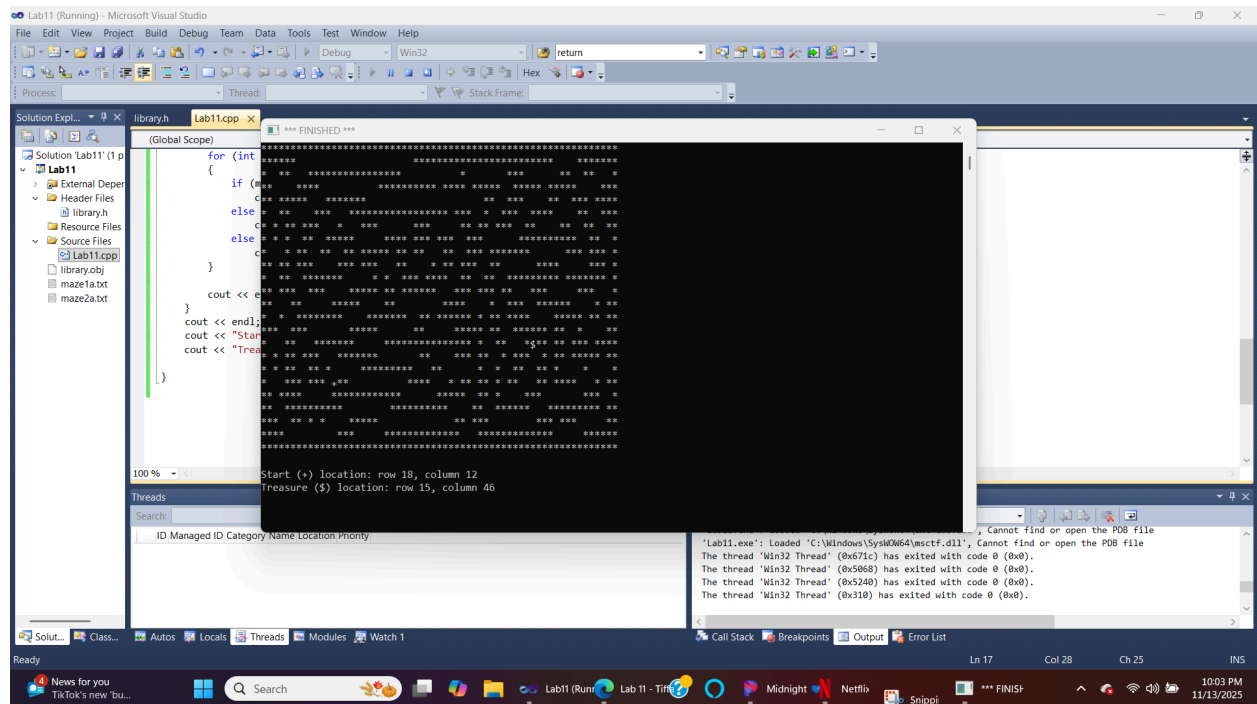
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        if (maze[r][c] == 'X')
            cout << "**";
        else if (maze[r][c] == '-')
            cout << " ";
        else
            cout << maze[r][c];
    }

    cout << endl;
}

    cout << endl;
    cout << "Start (+) location: row " << start_row << ", column " << start_column << endl;
    cout << "Treasure ($) location: row " << treasure_row << ", column " << treasure_column <<
endl;

}

```



3. Draw it Properly

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // start point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else set_pen_color(color::white);

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}
```

```
void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}
```

```
void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}
```

```
void main()
{
    char maze[size][size];
    int rows = 0;
    int columns = 0;
    int start_row = -1, start_column = -1;
    int treasure_row = -1, treasure_column = -1;

    ifstream fin("maze2a.txt");

    if (!fin)
    {
        cout << "Can't open file :(";
    }

    string line;

    while (getline(fin, line))
    {
        columns = (int)line.length();

        for (int c = 0; c < columns; c++)
        {
            maze[rows][c] = line[c];
        }
    }
}
```

```

        if (line[c] == '+')
        {
            start_row = rows; start_column = c;
        }
        if (line[c] == '$')
        {
            treasure_row = rows; treasure_column = c;
        }
    }

    rows++;
}

fin.close();

make_window(columns * cell, rows * cell);
set_pen_width(1);

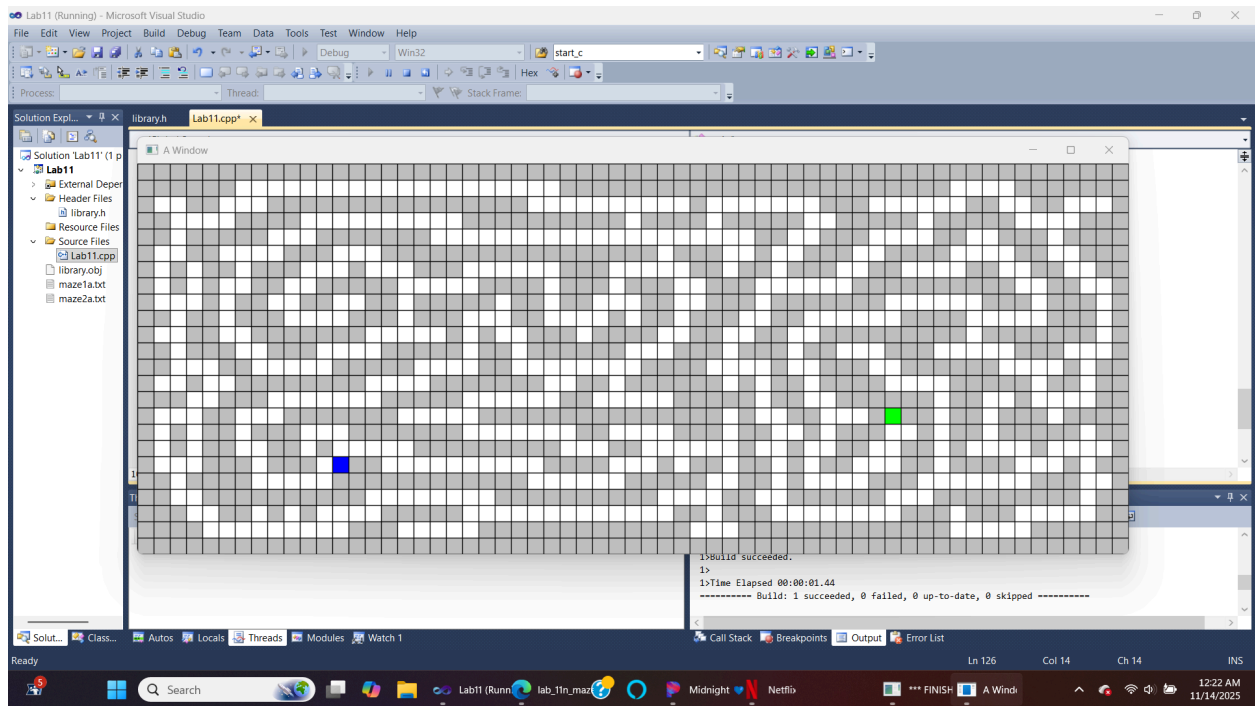
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell;
        int y = r * cell;

        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {
            moving_space(x, y, cell);
        }
    }
}

```

```
filled_square(start_column * cell, start_row * cell, cell, 2);  
}
```

// note: the robot square (blue) is overlapping with the starting square (red)



4. Make the Robot Move

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;

const int size = 80;
const int cell = 20;

void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}

void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else set_pen_color(color::white);

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}
```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Try to make it to the treasure :) (green sqaure)" <<
endl;

```

```

    char maze[size][size];
    int rows = 0;
    int columns = 0;
    int start_row = -1, start_column = -1;
    int treasure_row = -1, treasure_column = -1;

```

```

    ifstream fin("maze2a.txt");

```

```

    if (!fin)
    {
        cout << "Can't open file :(";
    }

```

```

    string line;

```

```

    while (getline(fin, line))
    {
        columns = (int)line.length();
    }

```

```

for (int c = 0; c < columns; c++)
{
    maze[rows][c] = line[c];

    if (line[c] == '+')
    {
        start_row = rows; start_column = c;
    }
    if (line[c] == '$')
    {
        treasure_row = rows; treasure_column = c;
    }
}

rows++;
}

```

```

fin.close();

```

```

make_window(columns * cell, rows * cell);
set_pen_width(1);

```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell;
        int y = r * cell;

        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {

```

```

        moving_space(x, y, cell);
    }
}

filled_square(start_column * cell, start_row * cell, cell, 2);

int robot_row = start_row;
int robot_column = start_column;

while (true)
{
    char c = wait_for_key_typed();

    if (c == 'q')
    {
        break;
    }

    if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
    else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
    else
    {
        moving_space(robot_column * cell, robot_row * cell, cell);
    }

    if (c == -91)
    {
        robot_column--;
    }
    if (c == -89)
    {
        robot_column++;
    }
    if (c == -90)
    {
        robot_row--;
    }
}

```

```
if (c == -88)
    {
        robot_row++;
    }

    filled_square(robot_column * cell, robot_row * cell, cell, 2);
}
```

// note: the robot square (blue) is overlapping with the starting square (red)

[Link to screen recording of moving robot](#)

5. Prevent Walking Through Walls

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;

const int size = 80;
const int cell = 20;

void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}

void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else set_pen_color(color::white);

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}
```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Try to make it to the treasure :) (green sqaure)" <<
endl;

```

```

    char maze[size][size];
    int rows = 0;
    int columns = 0;
    int start_row = -1, start_column = -1;
    int treasure_row = -1, treasure_column = -1;

```

```

    ifstream fin("maze2a.txt");

```

```

    if (!fin)
    {
        cout << "Can't open file :(";
    }

```

```

    string line;

```

```

    while (getline(fin, line))
    {
        columns = (int)line.length();
    }

```

```

for (int c = 0; c < columns; c++)
{
    maze[rows][c] = line[c];

    if (line[c] == '+')
    {
        start_row = rows; start_column = c;
    }
    if (line[c] == '$')
    {
        treasure_row = rows; treasure_column = c;
    }
}

rows++;
}

```

```

fin.close();

```

```

make_window(columns * cell, rows * cell);
set_pen_width(1);

```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell;
        int y = r * cell;

        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {

```

```

        moving_space(x, y, cell);
    }
}

filled_square(start_column * cell, start_row * cell, cell, 2);

int robot_row = start_row;
int robot_column = start_column;

while (true)
{
    char c = wait_for_key_typed();

    int new_row = robot_row;
    int new_column = robot_column;

    if (c == 'q')
    {
        break;
    }
    if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
    else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
    else
    {
        moving_space(robot_column * cell, robot_row * cell, cell);
    }
    if (c == -91)
    {
        new_column--;
    }
    if (c == -89)
    {
        new_column++;
    }
    if (c == -90)
    {

```

```

        new_row--;
    }
    if (c == -88)
    {
        new_row++;
    }
    if (maze[new_row][new_column] == 'X')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
        continue;
    }
    if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
    else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
    else
    {
        moving_space(robot_column * cell, robot_row * cell, cell);
    }

    robot_row = new_row;
    robot_column = new_column;

    filled_square(robot_column * cell, robot_row * cell, cell, 2);
}
}

```

// note: the robot square (blue) is overlapping with the starting square (red)

[Link to screen recording of moving robot \(that doesnt move through walls\)](#)

6. Automatic Mode

```
#include <iostream>
#include <fstream>
#include <string>
#include "library.h"
using namespace std;
```

```
const int size = 80;
const int cell = 20;
```

```
void square_outline(int x, int y, int s)
{
    set_pen_color(color::black);
    move_to(x, y);
    draw_to(x + s, y);
    draw_to(x + s, y + s);
    draw_to(x, y + s);
    draw_to(x, y);
}
```

```
void filled_square(int x, int y, int s, int which)
{
    if (which == 0)
    {
        set_pen_color(color::red); // starting point
    }
    else if (which == 1)
    {
        set_pen_color(color::green); // treasure yasss!!
    }
    else if (which == 2)
    {
        set_pen_color(color::blue); // robot
    }
    else set_pen_color(color::white);

    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}
```

```

void moving_space(int x, int y, int s)
{
    set_pen_color(color::white);
    fill_rectangle(x, y, s, s);
    square_outline(x, y, s);
}

```

```

void walls(int x, int y, int s)
{
    set_pen_color(color::grey);
    fill_rectangle(x, y, s, s);

    set_pen_color(color::grey);
    square_outline(x, y, s);
}

```

```

void main()
{
    cout << "Hello, Welcome to Maze Planet!" << endl << endl << "Use the arrows on your
keyboard to move." << endl << endl << "Press 'a' key to put the robot in automatic mode." <<
endl << endl << "Try to make it to the treasure :) (green square)" << endl;

```

```

    char maze[size][size];
    int rows = 0;
    int columns = 0;
    int start_row = -1, start_column = -1;
    int treasure_row = -1, treasure_column = -1;

```

```

    ifstream fin("maze2a.txt");

```

```

    if (!fin)
    {
        cout << "Can't open file :(";
    }

```

```

    string line;

```

```

    while (getline(fin, line))
    {
        columns = (int)line.length();
    }

```

```

for (int c = 0; c < columns; c++)
{
    maze[rows][c] = line[c];

    if (line[c] == '+')
    {
        start_row = rows; start_column = c;
    }
    if (line[c] == '$')
    {
        treasure_row = rows; treasure_column = c;
    }
}

rows++;
}

```

```

fin.close();

```

```

make_window(columns * cell, rows * cell);
set_pen_width(1);

```

```

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < columns; c++)
    {
        int x = c * cell;
        int y = r * cell;

        if (maze[r][c] == 'X')
        {
            walls(x, y, cell);
        }
        else if (maze[r][c] == '+')
        {
            filled_square(x, y, cell, 0);
        }
        else if (maze[r][c] == '$')
        {
            filled_square(x, y, cell, 1);
        }
        else
        {

```

```

        moving_space(x, y, cell);
    }
}

filled_square(start_column * cell, start_row * cell, cell, 2);

int robot_row = start_row;
int robot_column = start_column;

bool automatic = false;
const int moves = 11;

while (true)
{
    char c = 0;

    if (!automatic)
    {
        c = wait_for_key_typed();
    }
    else
    {
        for (int n = 0; n < moves; ++n)
        {
            int r = rand() % 4;

            if (r == 0)
            {
                c = -91;
            }

            if (r == 1)
            {
                c = -89;
            }

            if (r == 2)
            {
                c = -90;
            }

            if (r == 3)
            {
                c = -88;
            }
        }
    }
}

```

```

int new_row = robot_row;
int new_column = robot_column;

if (c == -91)
    {
        new_column--;
    }
if (c == -89)
    {
        new_column++;
    }
if (c == -90)
    {
        new_row--;
    }
if (c == -88)
    {
        new_row++;
    }
if (maze[new_row][new_column] == 'X')
{
    filled_square(robot_column * cell, robot_row * cell, cell, 2);
}
else
{
    if (maze[robot_row][robot_column] == '+')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 0);
        }
    else if (maze[robot_row][robot_column] == '$')
        {
            filled_square(robot_column * cell, robot_row * cell, cell, 1);
        }
    else
        {
            moving_space(robot_column * cell, robot_row * cell, cell);
        }
}

robot_row = new_row;
robot_column = new_column;

filled_square(robot_column * cell, robot_row * cell, cell, 2);

```

```

    }

    wait(0.15);
}

char k = wait_for_key_typed();

if (k == 'q')
    {
        break;
    }
if (k == 'a')
{
    automatic = false;
    continue;
}
else
{
    automatic = true;
    continue;
}
}

```

```

if (c == 'q')
    {
        break;
    }
if (c == 'a')
{
    automatic = true;
    continue;
}

```

```

int new_row = robot_row;
int new_column = robot_column;

```

```

if (c == -91)
    {
        new_column--;
    }
if (c == -89)
    {

```

```

        new_column++;
    }
    if (c == -90)
    {
        new_row--;
    }
    if (c == -88)
    {
        new_row++;
    }
    if (maze[new_row][new_column] == 'X')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 2);
        continue;
    }
    if (maze[robot_row][robot_column] == '+')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 0);
    }
    else if (maze[robot_row][robot_column] == '$')
    {
        filled_square(robot_column * cell, robot_row * cell, cell, 1);
    }
    else
    {
        moving_space(robot_column * cell, robot_row * cell, cell);
    }

    robot_row = new_row;
    robot_column = new_column;

    filled_square(robot_column * cell, robot_row * cell, cell, 2);

// did u win?
if (maze[robot_row][robot_column] == '$')
{
    make_window(700,250);
    set_pen_color(color::white);
    fill_rectangle(0, 0, 700, 250);
    set_pen_color(color::green);
    set_font_size(40);
    move_to(100, 150);
    write_string("YOU FOUND THE TREASURE!!");
}

```

```
}  
}  
}
```

// note: the robot square (blue) is overlapping with the starting square (red)

[Link to screen recording of robot moving automatically \(note: the robot moved automatically for the first few seconds before I switched it back to normal\)](#)